

Real World Examples

2026.04.17 강의 setuid0 / wooz3k



목차

I. Harvest Finance

II. Fei-Rari Protocol

III. Superfluid

Harvest Finance

Oracle manipulation attack

Oracle을 잘못 만들었거나 원래 Oracle로 쓰면 안 되는 것을 사용했을 때 발생
(예: Uniswap pool의 spot price, Curve pool의 토큰 비율)

Oracle은 주로 토큰의 가격을 알아볼 때 사용됨
(예: Lending 프로토콜에서 담보로 받은 ETH의 현재 USD 가치는 얼마?)

조작 가능한 Oracle을 사용한 대가는?...
TVL이 다 털려버리는 수가 있다. 담보의 가치를 실제보다 높게 평가 받고 안 값을
돈을 빌린다거나 해서...



Harvest Finance (\$33.8M)

Harvest Finance는 Curve를 stable coin의 price oracle로 사용

1. USDC를 USDT로 Swap (Curve에서 USDT의 가격 상승)
2. Harvest Finance에 Deposit (조작된 USDT 가격 기준으로 vault share 지급)
3. USDT를 USDC로 Swap (Curve에서 USDT의 가격 하락, 시세 조작에 든 비용 대부분 회수)
4. Harvest Finance에서 Withdraw
(vault share를 비정상적으로 많이 받았기 때문에 더 많은 USDT를 받아서 이득)
5. 반복

Arbitrage check는 풀 내부 가격이 최근 가격 대비 $\pm 3\%$ 초과 이탈 시 거부

→ 공격자는 임계 바로 아래로만 가격을 움직여 공격 반복

Harvest Finance Overview

Strategy Development

Harvest Finance team works hard every day to utilize the latest and most profitable techniques to bring returns to our users. New projects are relentlessly evaluated and pursued to increase our growing repertoire of yield farming strategies.

Every new project is evaluated through stages:

- Analyze project reputation
- Review Tokenomics - TVL, rewards, distribution, fees, deflation
- Audit smart contracts for safety
- Create a strategy for collecting and auto-compounding rewards
- Release and activate strategy - grant weekly FARM rewards

More info

Harvest takes care of gas costs, APY tracking, strategy development, and auditing.

We don't have deposit/withdrawal fees for any of our strategies.

A performance fee (30% ETH, 8% BSC) on profits is generated.

Harvest Finance?

- TradFi 기준: 유사 ETF
 - 전략 찾아서 토큰 넣어주는 컨트랙트임
 - 예:
 - X 프로토콜에 30% 넣고
Y 프로토콜에 70% 넣은 다음에,
주기적으로 claim해서 다시 invest를 해준다.
 - Performance fee로 reward의 30%를 떼감
- 1000 USDT 넣고 시간 지나면 1001USDT 빼가게 해야 함.

지분 accounting

- LP 토큰과 비슷한 식 사용

```
uint256 toMint = totalSupply() == 0  
    ? amount  
    : amount.mul(totalSupply()).div(underlyingBalanceWithInvestment());  
_mint(beneficiary, toMint);
```



underlyingBalanceWithInvestment()?

```
*/  
function underlyingBalanceWithInvestment() view public returns (uint256) {  
    if (address(strategy()) == address(0)) {  
        // initial state, when not set  
        return underlyingBalanceInVault();  
    }  
    return underlyingBalanceInVault().add(IStrategy(strategy()).investedUnderlyingBalance());  
}
```

```
/*  
 * Returns the cash balance across all users in this contract.  
 */  
function underlyingBalanceInVault() view public returns (uint256) {  
    return IERC20(underlying()).balanceOf(address(this));  
}
```

investedUnderlyingBalance()

```

1002  /**
1003  * Returns the underlying invested balance. This is the amount of yCRV that we are entitled to
1004  * from the yCRV vault (based on the number of shares we currently have), converted to the
1005  * underlying assets by the Curve protocol, plus the current balance of the underlying assets.
1006  */
1007  function investedUnderlyingBalance() public view returns (uint256) {
1008      uint256 shares = IERC20(ycrvVault).balanceOf(address(this));
1009      uint256 price = IVault(ycrvVault).getPricePerFullShare();
1010      // the price is in yCRV units, because this is a yCRV vault
1011      // the multiplication doubles the number of decimals for shares, so we need to divide
1012      // the precision is always 10 ** 18 as the yCRV vault has 18 decimals
1013      uint256 precision = 10 ** 18;
1014      uint256 ycrvBalance = shares.mul(price).div(precision);
1015      // now we can convert the balance to the token amount
1016      uint256 ycrvValue = underlyingValueFromYCrV(ycrvBalance);
1017      return ycrvValue.add(IERC20(underlying).balanceOf(address(this)));
1018  }
1019

```

Calculating Expected Amounts

DepositZap.**calc_withdraw_one_coin**(*_pool: address, _token_amount: uint256, i: int128*)→ uint256:
view

Calculate the amount received when withdrawing and unwrapping in a single coin. Useful for setting `_max_burn_amount` when calling `remove_liquidity_one_coin`.

- `_pool`: Address of the pool to deposit into.
- `_token_amount`: Amount of LP tokens to burn in the withdrawal.
- `i`: Index value of the underlying coin to withdraw.

Returns the expected amount of coin received.



Flash borrow 50M USDT and 10M USDC

Swap 10M USDC to USDT

Deposit 50M USDT &
Receive 52M pool tokens

Swap 10M USDT to USDC

Withdraw 50.5M USDT with 52M pool tokens

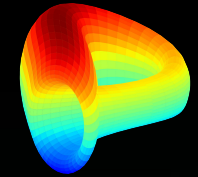
Repay flash loans
⇒ \$500K profit

Increases price of USDT

Query USDT price

Decreases price of USDT

Query USDT price



Curve
yUSD pool

Harvest Finance Proof of Concept Code 작성

UPSide Academy



과거의 공격 PoC 만드는 방법

- 공격자의 주소, TX 갖고오기
 - Address: <https://etherscan.io/address/0xf224ab004461540778a914ea397c589b677e27bb>
 - TX: <https://etherscan.io/tx/0x35f8d2f572fcea9c9288e5d462117850ef2694786992a8c3f6d02612277b0877>
- 공격 대상 Address 수집
 - Harvest FARM_USDT:
<https://etherscan.io/tx/0xa9e80d1214eb369ca443f4a025d7605914f758bc81036043120274fdd772ae2b>
 - ^ Logic Address at that time: 0x9b3be0cc5dd26fd0254088d03d8206792715588b

UPSide Academy



Proxy Contract ?! - 그 당시의 implementation 주소 찾기

<https://etherscan.io/tx/0xa9e80d1214eb369ca443f4a025d7605914f758bc81036043120274fdd772ae2b>

✓ call_0_1_5_9	0x45f783cce6b7ff23b2a...	→	⇒ 0xdac17f958d2ee523a2...	0 Ether	8,338,530
✓ staticcall_0_1_6	0xc6028a9fa486f52efd2...	→	⇒ 0xdac17f958d2ee523a2...	0 Ether	8,468,369
✓ call_0_1_7	0xc6028a9fa486f52efd2...	→	⇒ 0x053c80ea73dc6941f5...	0 Ether	8,464,393
✓ delegatecall_0_1_7_0	0x053c80ea73dc6941f5...	→	⇒ 0x9b3be0cc5dd26fd025...	0 Ether	8,330,451
✓ staticcall_0_1_7_0_0	0x053c80ea73dc6941f5...	→	⇒ 0xc95cbe4ca30055c787...	0 Ether	8,197,633
✓ staticcall_0_1_7_0_1	0x053c80ea73dc6941f5...	→	⇒ 0x222412af183bceadef...	0 Ether	8,194,891
✓ staticcall_0_1_7_0_2	0x053c80ea73dc6941f5...	→	⇒ 0x1c47343ea7135c2ba3...	0 Ether	8,190,292
✓ staticcall_0_1_7_0_2_0	0x1c47343ea7135c2ba3...	→	⇒ 0xd8d6ab3d2094d3a025...	0 Ether	8,058,081

언제로 돌아가지?

Initial Fund: tornado cash → 저 시점을 기준으로 block fork를 뜨면 됨. (-1)

0x6590d10cf772749e3a...	11129484	711 days 4 hrs ago	0xc6028a9fa486f52efd2...	→	Harvest.Finance: Hacker 1	20 Ether
0x2766f4219ab2439174...	11129483	711 days 4 hrs ago	0xc6028a9fa486f52efd2...	→	Harvest.Finance: Hacker 1	20 Ether
0x8ccfae48da0389f794a...	11129481	711 days 4 hrs ago	0xc6028a9fa486f52efd2...	→	Harvest.Finance: Hacker 1	20 Ether
0x81b459c1b20db1332d...	11129480	711 days 4 hrs ago	0xc6028a9fa486f52efd2...	→	Harvest.Finance: Hacker 1	20 Ether
0x7f7e8e55a231010833...	11129478	711 days 4 hrs ago	0xc6028a9fa486f52efd2...	→	Harvest.Finance: Hacker 1	20 Ether
0xa5f39df4ef609ef4a7a0...	11129477	711 days 4 hrs ago	0xc6028a9fa486f52efd2...	→	Harvest.Finance: Hacker 1	20 Ether
0x35f8d2f572fcea928...	11129474	711 days 4 hrs ago	0xc6028a9fa486f52efd2...	→	Harvest.Finance: Hacker 1	20 Ether
0x4b7b9e387a79289720...	11128634	711 days 7 hrs ago	Tornado.Cash: 10 ETH	→	Harvest.Finance: Hacker 1	9.984 Ether

foundry.toml 수정

- 아래 두 줄 추가
 - `eth_rpc_url = "[ETH RPC URL]"`
 - `fork_block_number = 11128633`

Upside Academy



PoC Step to reproduce

1. Flash loan USDT & USDC
2. Exchange USDC -> USDT on yCurve (Pump USDT)
3. Deposit USDT to Harvest FARM_USDT
4. Exchange USDT -> USDC on yCurve (Dump USDT)
5. Withdraw shares from the third step
6. Repay flash loan
7. Iterate 1-6 N times
8. Profit!



PoC 작성을 위한 주소들

- `h = HVault(0x053c80eA73Dc6941F518a68E2FC52Ac45BDE7c9C);`
- `c = CurveStrategy(0x1C47343eA7135c2bA3B2d24202AD960aDaFAa81c);`
- `y = yCurve(0x45F783CCE6B7FF23B2ab2D70e416cdb7D6055f51);`
- `uUSDT = UniSwapV2(0x0d4a11d5EEaaC28EC3F61d100daF4d40471f1852);`
- `uUSDC = UniSwapV2(0xB4e16d0168e52d35CaCD2c6185b44281Ec28C9Dc);`
- `usdt = ERC20(0xdAC17F958D2ee523a2206206994597C13D831ec7);`
- `usdc = ERC20(0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48);`

yCurve exchange

<https://etherscan.io/address/0x9b3be0cc5dd26fd0254088d03d8206792715588b>

```
StableSwap.exchange(i: int128, j: int128, _dx: uint256, _min_dy: uint256) → uint256
```

Perform an exchange between two coins.

- `i`: Index value for the coin to send
- `j`: Index value of the coin to receive
- `_dx`: Amount of `i` being exchanged
- `_min_dy`: Minimum amount of `j` to receive

Returns the actual amount of coin `j` received. Index values can be found via the `coins` public getter method.

```
>>> expected = pool.get_dy(0, 1, 10**2) * 0.99
>>> pool.exchange(0, 1, 10**2, expected, {"from": alice})
```



```
===== Final =====  
USDT Balance: 90960653809  
USDC Balance: 15171262945  
profit : 106131
```

Fei-Rari Protocol (~\$80M)

·} 돌아온 Reentrancy

잘 알려진 심각한 버그 클래스

최초의 케이스(The DAO)이기도 하고
그 이후에도 꽤 많이 발생했는데 잘 알려져 있고
좋은 Mitigation(ReentrancyGuard)이 있기 때문에
실상 있어서는 안되는 취약점인데...

Reentrancy 위협이 아예 없다고 가정하고 열어놨는데
알고보니 있어서 당해버리는 경우가 생김



Checks Effects Interactions 패턴

스마트 컨트랙트 함수 안에서 코드 순서를..

조건 체크

Storage 업데이트

그 다음에 External Call / Transfer / Send 등등 하라는 것

Reentrancy 자체가 발생하지 않는건 아니지만 공격이 불가능하게 됨

Reentrancy Guard

Global Lock 같은 느낌

nonReentrant modifier가 붙은 함수를 실행 중일때는
같은 컨트랙트의 모든 nonReentrant 함수 진입이 불가능해짐

Upside Academy



Compound fork인데..

알고보니 Compound에서 callback 있는 토큰 사용하면 Reentrancy가 발생할 수 있었다
(나중에는 절대 안되게 패치함)

아니 그러면 Compound는 왜 멀쩡하냐?

Compound 개발자들은 자기들 코드를 잘 이해하고 있어서 Governance를 통해서
Callback이 있는 토큰은 상장해주지 않는 절차가 있었음

Compound의 CEther는?

call이 아니라 transfer를 써서 가스 부족으로 공격이 안됐었음



Compound doTransferOut — Fei-Rari에서 transfer가 call로 바뀐 지점

```

1855
1856 ///////////////////////////////////////////////////
1857 // EFFECTS & INTERACTIONS
1858 // (No safe failures beyond this point)
1859
1860 + /*
1861 +  * We write the previously calculated values into storage.
1862 +  * Note: Avoid token reentrancy attacks by writing increased borrow before external transfer.
1863 +  */
1864 + accountBorrows[borrower].principal = accountBorrowsNew;
1865 + accountBorrows[borrower].interestIndex = borrowIndex;
1866 + totalBorrows = totalBorrowsNew;
1867 ✓ +
1868 ✓ /*
1869  * We invoke doTransferOut for the borrower and the borrowAmount.
1870  * Note: The cToken must handle variations between ERC-20 and ETH underlying.
1871 ✓ @@ -784,98 +608,55 @@ contract CToken is CTokenInterface, Exponential, TokenErrorReporter {
1872  */
1873  doTransferOut(borrower, borrowAmount);
1874
1875 - /* We write the previously calculated values into storage */
1876 - accountBorrows[borrower].principal = vars.accountBorrowsNew;
1877 - accountBorrows[borrower].interestIndex = borrowIndex;
1878 - totalBorrows = vars.totalBorrowsNew;
1879 ✓ -
1880 /* We emit a Borrow event */
1881 - emit Borrow(borrower, borrowAmount, vars.accountBorrowsNew, vars.totalBorrowsNew);
1882 -
1883 - /* We call the defense hook */
1884 - comptroller.borrowVerify(address(this), borrower, borrowAmount);
1885 -
1886 - return uint(Error.NO_ERROR);
1887 ✓ + emit Borrow(borrower, borrowAmount, accountBorrowsNew, totalBorrowsNew);
1888 }

```



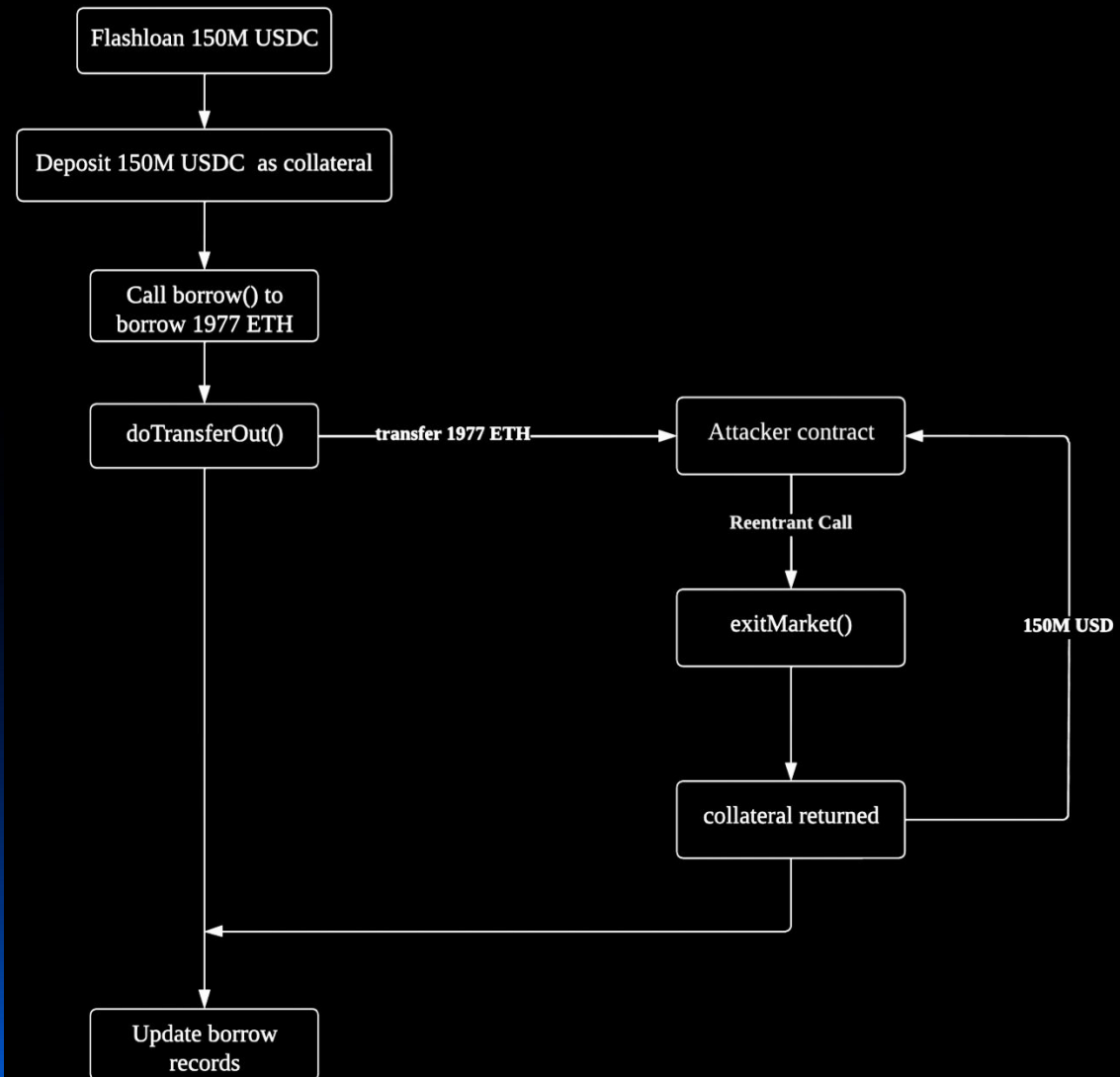
아무 생각 없이 CEther에 transfer로 돼있던 것을 call로 바꿔버림

ETH 또는 Native Token을 빌리고

그것에 대한 storage 업데이트가 되기 전에 exitMarket을 해서 담보를 unlock해버림
(storage상으로는 빌린게 없는 상태니까 그냥 가져갈 수 있다)

<https://rekt.news/fei-rari-rekt/>

140	140		<code>function doTransferOut(address payable to, uint amount) internal {</code>
141		-	<code>/* Send the Ether, with minimal gas and revert on failure */</code>
142		-	<code>to.transfer(amount);</code>
	141	+	<code>// Send the Ether and revert on failure</code>
	142	+	<code>(bool success,) = to.call.value(amount)("");</code>
	143	+	<code>require(success, "doTransferOut failed");</code>



PoC 작성을 위한 주소들

<https://etherscan.io/address/0x6162759edad730152f0df8115c698a42e666157f>

```
IERC20 DAI = IERC20(0x6B175474E89094C44Da98b954EedeAC495271d0F);  
ComptrollerInterface unitroller = ComptrollerInterface(0xc54172e34046c1653d1920d4033Dd358c7a1aF4);  
CErc20Interface fDAI = CErc20Interface(0x7e9cE3CAa9910cc048590801e64174957Ed41d43);  
CErc20Interface fETH = CErc20Interface(0xbB025D470162CC5eA24daF7d4566064EE7f5F111);
```

UPSide Academy

Superfluid

Upside Academy

Superfluid (-\$8.7M)

<https://rekt.news/superfluid-rekt/>

\$8.7M drained from Superfluid.

The crypto streaming protocol was hacked at 06:17 +UTC 08/02/22.

Other projects who were using the service for contributor payments or investor escrow contracts suffered negative price impacts as the attacker dumped their native tokens.

The affected protocols included Mai Finance (QI), Stacker Ventures (STACK), Stake DAO (SDT), and Museum of Crypto Art (MOCA).

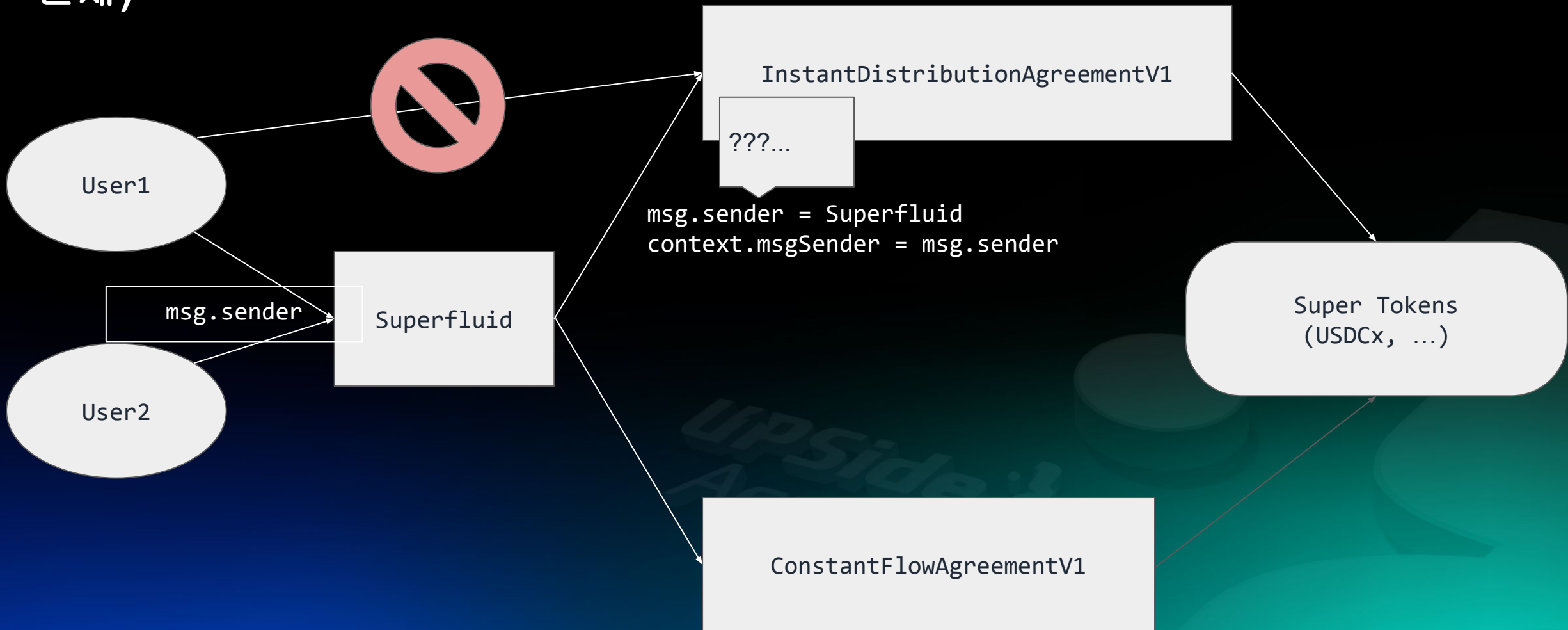
QI was the worst affected token, dropping initially by almost 80% following the dump, but it has since recovered to ~62% of its pre-hack price.

Superfluid's @francescorenzia told rekt.news that the attack was focussed only on the platform's larger wallet balances. In the time before the vulnerability was patched, the hacker left plenty of ETH, USDC and DAI untouched, presumably because the attacker had "had enough".

How did it happen?



Internal TX — agreement는 반드시 Superfluid host 경유(= context 신뢰의 전제)





외부 진입점 callAgreement

```
function callAgreement(  
    ISuperAgreement agreementClass,  
    bytes memory callData,  
    bytes memory userData  
)  
  
    external override  
    returns(bytes memory returnedData)  
{  
    return _callAgreement(msg.sender, agreementClass, callData, userData);  
}
```

_callAgreement — 내부 dispatcher

```
function _callAgreement(
    address msgSender,
    ISuperAgreement agreementClass,
    bytes memory callData,
    bytes memory userData
)
{
    internal
    cleanCtx
    isAgreement(agreementClass)
    returns(bytes memory returnedData)
{
    // beware of the endiness
    bytes4 agreementSelector = CallUtils.parseSelector(callData);

    //Build context data
    bytes memory ctx = _updateContext(Context({
        appLevel: isApp(ISuperApp(msgSender)) ? 1 : 0,
        callType: ContextDefinitions.CALL_INFO_CALL_TYPE_AGREEMENT,
        /* solhint-disable-next-line not-rely-on-time */
        timestamp: block.timestamp,
        msgSender: msgSender,
        agreementSelector: agreementSelector,
        userData: userData,
        appAllowanceGranted: 0,
        appAllowanceWanted: 0,
        appAllowanceUsed: 0,
        appAddress: address(0),
        appAllowanceToken: ISuperfluidToken(address(0))
    }));
    bool success;
    (success, returnedData) = _callExternalWithReplacedCtx(address(agreementClass), callData, ctx);
    if (!success) {
        revert(CallUtils.getRevertMsg(returnedData));
    }
    // clear the stamp
    _ctxStamp = 0;
}
```



_updateContext — 정상 흐름에서의 ctx 생성·할당

```
function _updateContext(Context memory context)
    private
    returns (bytes memory ctx)
{
    require(context.appLevel <= MAX_APP_LEVEL, "SF: APP_RULE_MAX_APP_LEVEL_REACHED");
    uint256 callInfo = ContextDefinitions.encodeCallInfo(context.appLevel, context.callType);
    uint256 allowanceIO =
        context.appAllowanceGranted.toUint128() |
        (uint256(context.appAllowanceWanted.toUint128()) << 128);
    ctx = abi.encode(
        abi.encode(
            callInfo,
            context.timestamp,
            context.msgSender,
            context.agreementSelector,
            context.userData
        ),
        abi.encode(
            allowanceIO,
            context.appAllowanceUsed,
            context.appAddress,
            context.appAllowanceToken
        )
    );
    _ctxStamp = keccak256(ctx);
}
```



_callExternalWithReplacedCtx — ctx 교체 후 대상 호출

```
function _callExternalWithReplacedCtx(
    address target,
    bytes memory callData,
    bytes memory ctx
)
    private
    returns(bool success, bytes memory returnedData)
{
    assert(target != address(0));

    // STEP 1 : replace placeholder ctx with actual ctx
    callData = _replacePlaceholderCtx(callData, ctx);

    // STEP 2: Call external with replaced context
    // FIXME make sure existence of target due to EVM rule
    /* solhint-disable-next-line avoid-low-level-calls */
    (success, returnedData) = target.call(callData);

    if (success) {
        require(returnedData.length > 0, "SF: APP_RULE_CTX_IS_EMPTY");
    }
}
```



replacePlaceholderCtx — trailing ABI 영역 교체 로직

```
function _replacePlaceholderCtx(bytes memory data, bytes memory ctx)
    internal pure
    returns (bytes memory dataWithCtx)
{
    // 1.a ctx needs to be padded to align with 32 bytes bundary
    uint256 dataLen = data.length;

    // double check if the ctx is a placeholder ctx
    {
        uint256 placeHolderCtxLength;
        // solhint-disable-next-line no-inline-assembly
        assembly { placeHolderCtxLength := mload(add(data, dataLen)) }
        require(placeHolderCtxLength == 0, "SF: placerholder ctx should have zero length");
    }

    // 1.b remove the placeholder ctx
    // solhint-disable-next-line no-inline-assembly
    assembly { mstore(data, sub(dataLen, 0x20)) }

    // 1.c pack data with the replacement ctx
    return abi.encodePacked(
        data,
        // bytes with padded length
        uint256(ctx.length),
        ctx, new bytes(CallUtils.padLength32(ctx.length) - ctx.length) // ctx padding
    );
}
```



<https://docs.soliditylang.org/en/latest/abi-spec.html#function-selector-and-argument-encoding>

함수가 받으려고 하는 인자 뒤에 추가로 뭔가 있어도 무시하고 잘 작동함

UPSide Academy



```
superfluid.callAgreement(  
    ida,  
    abi.encodeWithSelector(  
        InstantDistributionAgreementV1.createIndex.selector,  
        address(token),  
        indexId,  
        new bytes(0) ctx로 바뀜  
    ),  
    new bytes(0)  
);
```

```
function createIndex(  
    ISuperfluidToken token,  
    uint32 indexId,  
    bytes calldata ctx  
)
```

```
superfluid.callAgreement(  
    ida,  
    abi.encodeWithSelector(  
        InstantDistributionAgreementV1.createIndex.selector,  
        address(token),  
        indexId,  
        encodeCtx(fakeContext),  
        new bytes(0) ctx로 바뀔  
    ),  
    new bytes(0)  
);
```

```
function createIndex(  
    ISuperfluidToken token,  
    uint32 indexId,  
    bytes calldata ctx  
)
```

어?...



```
ISuperfluid.Context memory context = AgreementLibrary.authorizeTokenAccess(token, ctx);  
address publisher = context.msgSender;  
bytes32 iId = _getPublisherId(publisher, indexId);  
require(!_hasIndexData(token, iId), "IDA: E_INDEX_EXISTS");  
  
token.createAgreement(iId, _encodeIndexData(IndexData(0, 0, 0)));  
  
emit IndexCreated(token, publisher, indexId, context.userData);  
  
// nothing to be recorded so far  
newCtx = ctx;
```

2. 아아.. 망했어요..

1. 여기서 뭔가 체크할 것 같지만 안함



<https://polygonscan.com/tx/0xdee86cae2e1bab16496a49b2ec61aae0472a7ccf06f79744d42473e96edd6af6>

UPSide Academy

context.msgSender를 조작해서 할 수 있는 것?

msg.sender를 조작한 것과 비슷한 효과.

Superfluid는 asset streaming protocol이니 잘 이용하면 자금 탈취 가능

<https://github.com/superfluid-finance/protocol-monorepo/blob/2366f30f93ced16aea43534732600da08140c01e/packages/ethereum-contracts/contracts/agreements/InstantDistributionAgreementV1.sol>
!

[HINT] createIndex -> updateSubscription -> updateIndex -> claim -> token.downgrade



PoC 작성을 위한 주소들

superfluid = 0x3E14dC1b13c488a8d5D310918780c983bD5982E7

ida = 0xB0aABBA4B2783A72C52956CDEF62d438ecA2d7a1

superusdc = 0xCAa7349CEA390F89641fe306D93591f87595dc1F

usdc = 0x2791Bca1f2de4661ED88A30C99A7a9449Aa84174

victim = ?

victim = 0x2e9e3C24049655f2D8C59f08602Da3DE4aD34188

fork_block_number = 24684668



<https://github.com/superfluid-finance/protocol-monorepo/commit/133472a40ebe5b6a7d7e16679b0b4c554f98ab43>

이것도 완벽하지는 않았음

```
39      39      function authorizeTokenAccess(ISuperfluidToken token, bytes memory ctx)
40      40      internal view
41      41      returns (ISuperfluid.Context memory)
42      42      {
43      -      require(token.getHost() == msg.sender, "AgreementLibrary: unauthroized host");
43      +      require(token.getHost() == msg.sender, "unauthorized host");
44      +      require(ISuperfluid(msg.sender).isCtxValid(ctx), "invalid ctx");
44      45      return ISuperfluid(msg.sender).decodeCtx(ctx);
45      46      }
```



```
function claim(
    ISuperfluidToken token,
    address publisher,
    uint32 indexId,
    address subscriber,
    bytes calldata ctx
)
    external override
    returns(bytes memory newCtx)
{
    _SubscriptionOperationVars memory vars;
    AgreementLibrary.CallbackInputs memory cbStates;

    (
        vars.iId,
        vars.sId,
        vars.idata,
        ,
        vars.sdata
    ) = _loadAllData(token, publisher, subscriber, indexId, true);

    // required condition check
    require(vars.sdata.subId == _UNALLOCATED_SUB_ID, "IDA: E_SUBS_APPROVED");

    uint256 pendingDistribution = uint256(vars.idata.indexValue - vars.sdata.indexValue)
        * uint256(vars.sdata.units);

    cbStates = AgreementLibrary.createCallbackInputs(
        token,
        publisher,
        vars.sId,
        "");
    newCtx = ctx;

    if (pendingDistribution > 0) {
        cbStates.noopBit = SuperAppDefinitions.BEFORE_AGREEMENT_UPDATED_NOOP;
        vars.cbdata = AgreementLibrary.callAppBeforeCallback(cbStates, newCtx);
    }
}
```

과제 예고

InstantDistributionAgreementV1.claim 취약점 익스플로잇 작성

context.msgSender는 덮어쓰워져서 이용할 수 없지만

context의 다른 필드를 조작하는 더 강력한 공격으로
SuperToken의 underlying 토큰을 모두 가져올 수 있음

패치 2

<https://github.com/superfluid-finance/protocol-monorepo/commit/84f366b3d30d242d0a9173ced45b0db227222cb3>

```
776      776      function claim(  
777      777          ISuperfluidToken token,  
778      778          address publisher,  
779      779          uint32 indexId,  
780      780          address subscriber,  
781      781          bytes calldata ctx  
782      782      )  
783      783      external override  
784      784      returns(bytes memory newCtx)  
785      785      {  
786      786      +      AgreementLibrary.authorizeTokenAccess(token, ctx);  
787      787      +  
786      788      _SubscriptionOperationVars memory vars;  
787      789      AgreementLibrary.CallbackInputs memory cbStates;
```